

Low-Resource Armenian NLP: Tokenization Surgery and Lightweight Adaptation

Albert Hakobyan

*Zaven and Sonia Akian College of Science
and Engineering, American University of Armenia*
albert_hakobyan@edu.aua.am

Robert Gadukyan

*Zaven and Sonia Akian College of Science
and Engineering, American University of Armenia*
robert_gadukyan@edu.aua.am

Levon Gevorgyan

*Zaven and Sonia Akian College of Science
and Engineering, American University of Armenia*
levon_gevorgyan@edu.aua.am

Silva Vardanyan

*Zaven and Sonia Akian College of Science
and Engineering, American University of Armenia*
silva_vardanyan@edu.aua.am

Dr. Erik Arakelyan

*American University of Armenia,
NVIDIA*
erik.arakelyan@aua.am

Abstract—This work investigates whether the poor efficiency of English-centric and broadly multilingual tokenizers on Armenian can be corrected by training an Armenian-specific tokenizer, grafting it onto a small pretrained language model, and recovering model behavior with lightweight fine-tuning. The pipeline consists of baseline tokenizer fertility analysis, custom tokenizer training, vocabulary grafting with alternative embedding initialization strategies, LoRA recovery fine-tuning, and final evaluation with generation and preservation diagnostics. Experimental results showed that tokenizer fertility ranged from 2.18 tokens per Armenian word for XLM-R to 14.26 for GPT-2, a $6.5\times$ spread. Qwen2.5-0.5B, the selected grafting target, used 7.83 tokens per Armenian word and had no Armenian-specific vocabulary tokens in the project count. Six Armenian SentencePiece tokenizers were trained on a five-million-line sample from the cleaned CC-100 Armenian corpus. All six beat XLM-R; the best, `hy_bpe_32k`, reached 1.67 tokens per word, 8.72 bytes per token, 0.683 single-token retention rate, 2.5% severe fragmentation, and 0% UNK rate under the Armenian-filtered, byte-fallback evaluation setup. Grafting this tokenizer onto Qwen2.5-0.5B added 30,766 tokens and reduced Qwen’s Armenian fertility from 7.81 to 1.69 tokens per word on the Goal 3 sample. Heuristic/FOCUS embedding initialization gave the best pre-fine-tuning perplexity, 24,484.3, and remained best after LoRA recovery with evaluation perplexity 8.33. Qualitative generation remained weak, so the strongest defensible conclusion is tokenizer-efficiency and perplexity recovery, not complete downstream Armenian generation quality.

Index Terms—Armenian NLP, low-resource languages, tokenization, SentencePiece, vocabulary grafting, LoRA, embedding initialization

I. INTRODUCTION

Low-resource language NLP often focuses on model size, data volume, and downstream fine-tuning. This project ex-

amines a more basic bottleneck: tokenization. Armenian is written in its own script and has rich morphology. Tokenizers trained primarily for English, code, or high-resource multilingual mixtures often fail to represent Armenian words as meaningful subword units. Instead, they fall back to character-level or byte-level fragmentation. That increases context-window consumption, training cost, inference cost, and the difficulty of learning stable lexical and morphological representations.

The project goal was to test whether this bottleneck can be improved without pretraining a new language model from scratch. The chosen approach is tokenizer surgery: train a better Armenian tokenizer, graft its tokens onto a pretrained model, initialize the new embeddings with principled strategies, and run short recovery fine-tuning. The project was organized into five stages:

- 1) **Goal 1: Baseline tokenizer fertility analysis** – comparative fertility analysis of standard tokenizers on Armenian text;
- 2) **Goal 2: Armenian tokenizer training and evaluation** – training and evaluation of six Armenian SentencePiece tokenizers at 8k, 16k, and 32k vocabulary sizes;
- 3) **Goal 3: Vocabulary grafting and embedding initialization** – extension of Qwen2.5-0.5B with the best-performing tokenizer and comparison of mean, heuristic/FOCUS, and nearest-token embedding initialization strategies;
- 4) **Goal 4: LoRA recovery fine-tuning and qualitative evaluation** – recovery fine-tuning experiments evaluated through perplexity and qualitative generation behavior, together with discussion of incomplete artifacts and missing downstream evalua-

tion;

- 5) **Goal 5: Final evaluation and visualization** – consolidation of the complete pipeline into final result tables, publication figures, improved generation diagnostics, and an English preservation check.

II. RELATED WORK

The literature review focuses on tokenizer surgery, the FOCUS method, multilingual tokenization challenges, low-resource language adaptation, and related LLaMA-style vocabulary grafting work. The Goal 3 embedding initialization experiments were motivated by the FOCUS method of Dobler and de Melo [1], whose core idea was adapted for constructing heuristic initialization vectors for newly added Armenian tokens. The key idea is to initialize a new token by encoding its surface form with the old tokenizer and averaging the old subtoken embeddings. This gives the new token a starting point derived from the pretrained embedding geometry instead of assigning it a random or nearly identical vector.

Tokenizer training was performed using SentencePiece [2] for the custom Armenian tokenizer training. It compares this approach against explicitly multilingual tokenizers such as XLM-R [3] and mT5 [4], as well as English-centric or broad general-purpose tokenizers. Lightweight recovery uses LoRA [5]. Qwen2.5-0.5B is used as the grafting target because it is small, accessible, modern, and has poor Armenian coverage in the baseline analysis [6].

The empirical setting matches the low-resource tokenizer problem closely. A model can have useful pretrained weights but still process Armenian inefficiently if its tokenizer has no Armenian subwords. The baseline tokenizer analysis revealed the mGPT-Armenian anomaly: although the model was Armenian-fine-tuned, its tokenizer contained zero Armenian vocabulary tokens in the reported count and still fragmented Armenian text at 12.43 tokens per word [7]. This supports the methodological premise that fine-tuning model weights alone does not remove tokenization overhead.

III. TOKENIZER FERTILITY ANALYSIS

A. Data and Preprocessing

All baseline tokenizer experiments were conducted on the CC-100 Armenian `hy.txt` corpus, consisting of approximately 5.91 GB of text and 25.3 million lines. It reports a cleaned corpus of 12,129,443 lines after NFC normalization, HTML removal, URL removal, MD5 deduplication, an Armenian-character-ratio filter, and a minimum-length filter. The local repository does not contain the full cleaned corpus `hy_clean.txt`; it contains the evaluation sample `hy_sample_raw.txt` and a large training sample `hy_train_sample.txt`.

The large file was inspected safely with metadata, head/tail, word-count, and bounded sampling commands. It is UTF-8 Armenian text with CRLF line endings, not a binary checkpoint. It contains 5,000,000 lines,

1,847,478,691 bytes, and 132,249,254 whitespace-delimited words. A 2 MB prefix sample had an Armenian-character-to-alphabetic-character ratio of 0.991. Table I summarizes the local and referenced data.

TABLE I
DATASET ARTIFACTS INSPECTED IN THE REPOSITORY.

Artifact	Lines	Bytes	Role
<code>hy_sample_raw.txt</code>	30,000	7.58M	Evaluation sample for Goals 1–3.
<code>hy_train_sample.txt</code>	5,000,000	1.85B	Five-million-line tokenizer-training sample.
<code>hy_clean.txt</code>	12.13M reported	4.48GB reported	Full cleaned corpus referenced by notebooks but absent locally.

B. Metrics

The fertility script defines Armenian-safe preprocessing: Unicode NFC normalization, removal of control and formatting characters, HTML and URL removal, whitespace normalization, and filtering for Armenian-heavy lines. The main metrics are:

- *fertility*: total encoded tokens divided by whitespace-delimited alphabetic words;
- *bytes per token*: total UTF-8 bytes divided by encoded tokens;
- *STRR*: single-token retention rate, or the fraction of words encoded as one token;
- *severe fragmentation*: fraction of words with word-level fertility at least five;
- *HY/EN ratio*: Armenian fertility divided by English fertility for the same tokenizer;
- *vocabulary coverage*: count and percentage of vocabulary entries containing Armenian characters.

C. Baseline Results

Table II reports the final baseline tokenizer results compiled from the project artifacts and accompanying documentation [7].

The strongest baseline is XLM-R, which is expected because it is explicitly multilingual and trained with CC-100 data. The worst baseline is GPT-2, which effectively treats Armenian as byte-level text. Qwen2.5-0.5B was selected for surgery despite poor Armenian fertility because it is small, accessible, modern, and has zero Armenian tokens, which makes the graft clean and easy to evaluate. The results also show that vocabulary size alone is not sufficient: in the project report, LLaMA-3 has a larger vocabulary than LLaMA-2 but worse Armenian fertility.

IV. CUSTOM ARMENIAN TOKENIZER TRAINING

Goal 2 focused on training six Armenian SentencePiece tokenizers using BPE and Unigram algorithms at vocabulary sizes of 8k, 16k, and 32k [2], [8]. The training data was a 5,000,000-line sample from the cleaned Armenian corpus,

TABLE II

BASILINE TOKENIZER FERTILITY ON ARMENIAN TEXT. LOWER FERTILITY AND SEVERE FRAGMENTATION ARE BETTER; HIGHER BYTES/TOKEN AND STRR ARE BETTER.

Tokenizer	Type	Vocab	Arm. vocab	Fert.	B/Tok	STRR	Severe	HY/EN
XLM-R-base	Unigram SP	250,002	3,528	2.18	6.65	0.450	7.3%	1.62×
mT5-small	Unigram SP	250,100	2,274	2.85	5.09	0.117	8.4%	1.91×
Gemma-2-2B	BPE SP	256,000	246	4.49	3.23	0.100	43.3%	3.55×
Qwen2.5-0.5B	BPE	151,665	0	7.83	1.85	0.059	69.2%	6.07×
LLaMA-2-7B	BPE SP	32,000	29	8.53	1.70	0.004	79.9%	5.80×
Mistral-v0.3	BPE SP	32,768	28	8.57	1.69	0.004	79.9%	5.96×
mGPT-Armenian	BPE SP	100,000	0	12.43	1.17	0.003	82.3%	9.84×
LLaMA-3-8B	tiktoken BPE	128,256	0	12.46	1.17	0.005	82.3%	9.92×
GPT-2	BPE	50,257	0	14.26	1.02	0.004	84.7%	11.20×

TABLE III

VOCABULARY COMPOSITION OF THE SIX CUSTOM TOKENIZERS.

Model	Vocab	Armenian	Arm. %	Other
unigram_8k	8,000	7,230	90.4	509
unigram_16k	16,000	14,784	92.4	955
unigram_32k	32,000	29,562	92.4	2,177
bpe_8k	8,000	7,178	89.7	561
bpe_16k	16,000	14,685	91.8	1,054
bpe_32k	32,000	29,641	92.6	2,098

with seed 42. Character coverage was 0.9999, byte fallback was enabled, normalization was NFKC, digit splitting was enabled, and the maximum SentencePiece length was 24 bytes. The special tokens were `<pad>`, `<unk>`, `<s>`, `</s>`, and `<mask>`. The notebook explains that a 5M-line sample was chosen because SentencePiece loads training sentences into memory, making the full 12.1M-line corpus less practical on a 16GB local machine.

The six `.vocab` files have the expected line counts: 8,000, 16,000, or 32,000 entries. Table III summarizes the vocabulary composition of the trained tokenizers.

Although byte fallback is enabled, the vocabulary composition output counts zero active byte-fallback tokens in its `byte_tokens` category and 261 special/reserved items. The `.vocab` files themselves begin with the five special tokens and `<0x00>`-style byte entries, so the notebook labels should be interpreted as a project-specific breakdown rather than evidence that byte fallback was disabled.

All six custom tokenizers were evaluated on a held-out Armenian sample containing 25,611 evaluation lines and 536,051 words. The results are shown in Table IV. All six custom tokenizers beat XLM-R, the strongest baseline. The best model was `bpe_32k`, with fertility 1.67, bytes/token 8.72, STRR 0.683, severe fragmentation 2.5%, and 0% UNK rate.

Custom tokenizers were ranked using a weighted evaluation score. Fertility received the highest weight because reducing tokens per Armenian word was the primary goal of the project. STRR was weighted next because single-token word coverage directly reflects usable Armenian subword quality. Compression and vocabulary efficiency were included as secondary criteria to balance sequence efficiency against vocabulary growth. The final weighting

TABLE IV

CUSTOM TOKENIZER RESULTS COMPARED WITH SELECTED BASELINES.

Tokenizer	Fert.	B/Tok	STRR	Sev.	UNK
bpe_32k	1.67	8.72	0.683	2.5%	0.0000%
unigram_32k	1.68	8.67	0.682	2.7%	0.0000%
bpe_16k	1.83	7.97	0.611	3.5%	0.0000%
unigram_16k	1.83	7.95	0.620	3.9%	0.0000%
bpe_8k	2.06	7.08	0.518	5.2%	0.0000%
unigram_8k	2.06	7.06	0.538	6.0%	0.0000%
XLM-R	2.18	6.65	0.450	7.3%	0.0000%
mT5	2.85	5.09	0.117	8.4%	0.0000%
Gemma-2	4.49	3.23	0.100	43.3%	0.0000%
GPT-2	14.26	1.02	0.004	84.7%	0.0000%

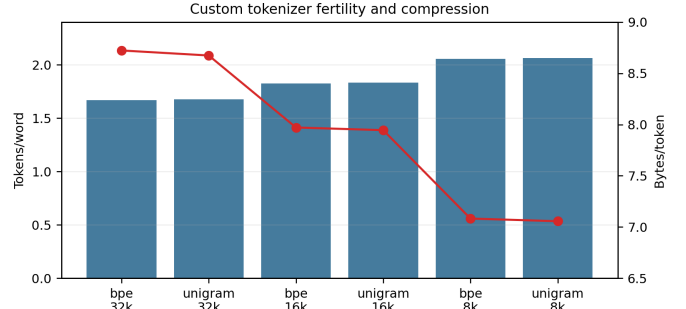


Fig. 1. Fertility and compression tradeoff for the six Armenian SentencePiece tokenizers. Larger vocabularies reduce tokens per word and increase bytes per token, while BPE and Unigram perform similarly at matched vocabulary sizes.

was 40% fertility, 30% STRR, 20% compression, and 10% vocabulary efficiency. This ranking selected `bpe_32k` with score 0.6442, narrowly ahead of `unigram_32k` at 0.6415. Figure 1 visualizes the same vocabulary-size tradeoff.

V. VOCABULARY GRAFTING AND EMBEDDING INITIALIZATION

Goal 3 focused on grafting the custom Armenian tokenizer onto Qwen/Qwen2.5-0.5B. The completed experiments evaluated multiple embedding initialization strategies and measured recovery through perplexity and qualitative generation behavior [9]. The grafting method used vocabulary extension rather than replacement. This preserves all old Qwen tokens and adds Armenian pieces from the custom tokenizer.

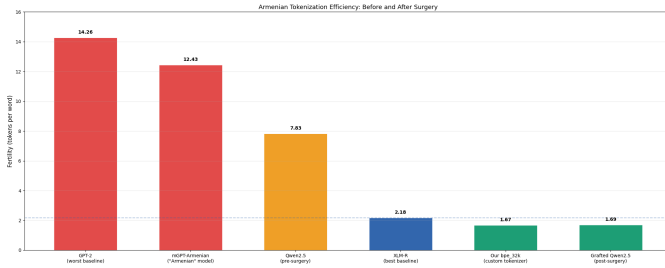


Fig. 2. End-to-end fertility comparison from Goal 5. English-centric and weakly covered baselines fragment Armenian heavily, while the custom `bpe_32k` tokenizer and its grafted Qwen variant reduce Armenian fertility to approximately 1.7 tokens per word.

The base model had 494,032,768 parameters. The original tokenizer length was 151,665, while the loaded model’s embedding table was reported as 151,936 by 896. The embeddings were tied. The Armenian tokenizer used for grafting was `hy_bpe_32k.model`. Of its 32,000 SentencePiece entries, 30,766 were added to Qwen, 973 were already present, and 256 byte/special tokens were skipped. The new-token breakdown was 29,641 Armenian tokens, 1,124 mixed/Latin tokens, and one space marker.

A. Fertility After Grafting

Fertility was evaluated on a 2,000-line Armenian sample during the Goal 3 experiments. Original Qwen produced 330,532 tokens over 42,325 words, or 7.81 tokens per word. The Armenian SP tokenizer produced 71,706 tokens over the same words, or 1.69 tokens per word. This is a 78.3% token-count reduction and a 4.6×reduction in tokens per Armenian word. Figure 2 summarizes the full fertility trajectory from the weakest baselines through the grafted tokenizer.

B. Initialization Strategies

The project compares three strategies for initializing the new Armenian token rows:

- *Mean initialization*: each new embedding is the mean of the old embedding table plus small Gaussian noise. It is stable but gives new tokens little lexical distinction.
- *Heuristic/FOCUS initialization*: each Armenian SentencePiece token is converted to a surface string, encoded by the old Qwen tokenizer, and initialized as the average of those old fallback-subtoken embeddings [1]. The notebook reports zero fallback-to-global-mean cases across the 30,766 new tokens.
- *Nearest-token initialization*: the same heuristic vector is computed, but the closest old embedding by cosine similarity is copied. This avoids averaging artifacts but maps many Armenian tokens to a small number of byte-like old tokens.

Table V reports embedding norm statistics. The nearest-token analysis found that 30,766 Armenian tokens

mapped to only 1,101 unique Qwen neighbor tokens, a 3.6% coverage ratio. This helps explain why nearest initialization may lose diversity despite having realistic norm magnitudes.

TABLE V
EMBEDDING NORM STATISTICS AFTER GRAFTING.

Group	Mean	Std.	Min	Max
Old Qwen tokens	0.4636	0.0611	0.2869	0.8066
Mean-init new	0.3494	0.0080	0.3169	0.3828
Heuristic-init new	0.2309	0.0545	0.1736	0.7160
Nearest-init new	0.4016	0.0444	0.2920	0.7160

C. Pre-Fine-Tuning Perplexity

Before recovery fine-tuning, Goal 3 evaluated each strategy on 300 Armenian lines with 10,995 tokens. Armenian text was segmented with the custom SP model and mapped into the extended Qwen vocabulary. Table VI shows that heuristic initialization started substantially closer to usable behavior than the other strategies.

TABLE VI
PRE-FINE-TUNING PERPLEXITY AFTER VOCABULARY GRAFTING.

Strategy	PPL	Tokens evaluated
Heuristic-init	24,484.3	10,995
Nearest-init	96,967.5	10,995
Mean-init	216,406.1	10,995

VI. RECOVERY FINE-TUNING

Recovery fine-tuning in Goal 4 was performed using LoRA/PEFT rather than full-model training [5], [10]. The reported setup used `hy_clean.txt`, 500,000 training lines, maximum sequence length 512, per-device batch size 4, gradient accumulation 8, effective batch size 32, one epoch, learning rate 2×10^{-4} , LoRA rank 16, LoRA alpha 32, and LoRA dropout 0.05 from the adapter config. The target modules were `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The PEFT task type was `CAUSAL_LM`, and adapter configs record PEFT version 0.19.1.

Different stages of the experimental pipeline were executed in separate software environments. Heuristic-init was trained on Google Colab T4 and took 277 minutes. Mean-init was run on a Vast.ai H100, while the final nearest-init run was executed on an RTX PRO 6000 workstation-class GPU. Therefore, runtime comparisons are hardware-confounded and should not be used as strategy-efficiency evidence.

Table VII combines the Goal 3 pre-fine-tuning perplexities with the Goal 4 recovery results. Figure 3 visualizes the same comparison before and after LoRA recovery, while Figure 4 shows the detailed training and evaluation loss trajectories during adaptation.

TABLE VII
LoRA RECOVERY FINE-TUNING RESULTS. THE NEAREST-INIT ROW REPORTS THE COMPLETED FINAL 2,425-STEP RUN.

Strategy	Pre-FT PPL	Post-FT PPL	Eval loss	Train loss	Steps	Hardware
Heuristic-init	24,484.3	8.33	2.1198	2.5141	2,425	Colab T4
Mean-init	216,406.1	10.09	2.3117	2.7720	2,425	H100
Nearest-init	96,967.5	8.96	2.1927	2.284	2,425	RTX PRO 6000

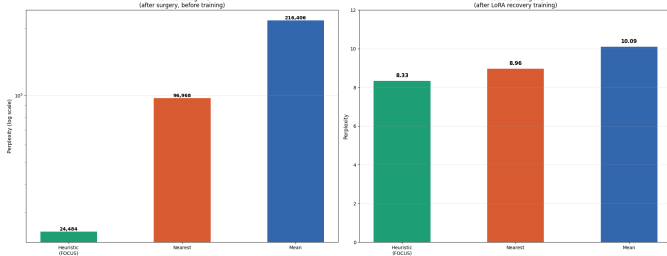


Fig. 3. Initialization strategy comparison from Goal 5. The heuristic/FOCUS initialization has the lowest pre-fine-tuning perplexity and remains the best strategy after LoRA recovery; nearest-token initialization improves substantially but remains behind heuristic initialization.

Heuristic initialization is best both before and after recovery fine-tuning. Mean initialization fully trains but remains worse. Nearest initialization also completes successfully in the final run and reaches 8.96 perplexity, but it still trails heuristic initialization.

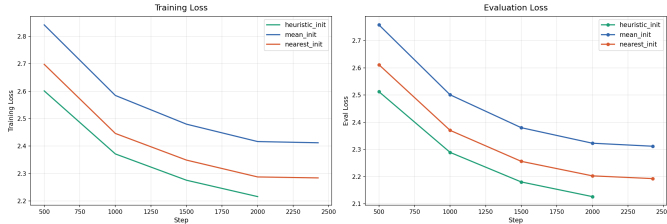


Fig. 4. Training and evaluation loss trajectories during LoRA recovery fine-tuning. Heuristic/FOCUS initialization converges faster and maintains the lowest losses across optimization steps, while nearest-token initialization consistently outperforms mean initialization after recovery begins.

VII. RESULTS AND DISCUSSION

A. Tokenization Quality

Among the baseline tokenizers, XLM-R performs best with fertility 2.18. Among custom tokenizers, **bpe_32k** performs best with fertility 1.67, closely followed by **unigram_32k** at 1.68. The difference between BPE and Unigram is small at each vocabulary size, contrary to the initial expectation that Unigram might clearly dominate because XLM-R and mT5 were the strongest baselines. The stronger predictor is vocabulary size: moving from 8k to 16k reduces fertility from 2.06 to 1.83, and moving from 16k to 32k reduces it further to 1.67–1.68.

The baseline table shows three fragmentation regimes. Explicitly multilingual SentencePiece tokenizers contain Armenian subwords and keep fertility below 3. Large general tokenizers such as Gemma and Qwen have partial or incidental coverage; Qwen has no Armenian vocabulary tokens in the project count and behaves near character/byte level. English-centric byte-level tokenizers such as GPT-2 and LLaMA-3 show severe fragmentation above 12 tokens per word.

B. Tokenizer Surgery Efficiency

Tokenizer surgery strongly improved efficiency. Qwen’s baseline fertility is 7.83 in Goal 1 and 7.81 in the Goal 3 sample. The grafted Armenian **bpe_32k** tokenizer reaches 1.69 tokens per word in the Goal 3 sample, a 78.3% token-count reduction. This means the same 512-token sequence budget can contain far more Armenian lexical content after grafting.

C. Embedding Initialization

Heuristic initialization wins because it uses the old tokenizer’s representation of the new token surface form. Even if Qwen fragments Armenian into byte-like pieces, those pieces are the only pretrained bridge between the old embedding space and the new Armenian tokens. Mean initialization gives nearly identical starts to all new tokens. Nearest initialization collapses many new tokens onto a small set of byte-like neighbors; the 3.6% unique-neighbor coverage ratio is evidence of this limitation.

D. Qualitative Tokenization

The tokenizer comparison in Goal 3 includes direct tokenization examples. Table VIII gives the token-count comparison. For the first sentence, original Qwen produces byte-like fragments such as $\tilde{\text{օ}}\tilde{\text{գ}}$, $\tilde{\text{օ}}_{\text{i}}$, and $\tilde{\text{օ}}_{\text{ք}}$, while the Armenian tokenizer produces word/subword pieces corresponding to Հայաստանը, Հարավային, Կովկասի, and երկիր. This is the desired effect: shorter and more linguistically interpretable inputs.

TABLE VIII
QUALITATIVE TOKENIZATION EXAMPLES FROM GOAL 3.

Text	Qwen toks.	Arm. SP toks.	Ratio
Հայաստանը Հարավային Կովկասի լեռնային երկիր է:	45	7	6.4×
Երևանը Հայաստանի մայրաքաղաքն ու խոշորագույն քաղաքն է:	53	9	5.9×
Ապագադուրացուն. 2-րդ մաս. Վարուժան Ավետիսյանի հոդվածը	53	15	3.5×

E. Generation Behavior

The final qualitative evaluation used the heuristic-initialized model together with the LoRA adapter on four Armenian prompts: Հայաստանը, Երևանը աշխարհի, Հայ ժողովուրդը, and Հայոց լեզուն. The continuations are mostly English or mixed multilingual text. Goal 5 repeated this evaluation with longer Armenian prompts, greedy decoding, short continuations, and top-token inspection. The longer prompts improved diagnostic control, but the generated continuations still drifted into English or mixed multilingual text. The defensible claim is therefore narrower than general-purpose Armenian generation: tokenization efficiency improved substantially, heuristic embedding initialization produced better perplexity before and after LoRA, and lightweight adaptation partially recovered next-token behavior as measured by held-out perplexity.

F. Goal 5 Final Evaluation

Goal 5 functions as an end-to-end evaluation layer over the earlier pipeline [11]. It consolidates the fertility, custom-tokenizer, grafting, and LoRA-recovery results into a single final results file; generates the pipeline visualizations in Figs. 2 and 3; and adds two lightweight diagnostics not covered by the core training stages. First, improved generation tests use longer Armenian prompts and deterministic decoding to reduce sampling noise. These tests confirm that recovery fine-tuning improves next-token likelihood without yet producing reliable Armenian continuations. Second, an English preservation check evaluates the heuristic-initialized recovered model on five English sentences. The adapted model obtains English perplexity 19.89, compared with 7.89 for the original Qwen2.5-0.5B under its native tokenizer. This indicates that the model did not collapse on English text, but English behavior is not fully preserved after vocabulary grafting and Armenian-focused adaptation.

G. Reproducibility and Artifact Issues

The experimental materials document the main workflow, including fertility analysis, tokenizer training, vocabulary grafting, and LoRA recovery fine-tuning. The trained tokenizers, grafted-model outputs, LoRA adapters, benchmark JSON files, and the plotted training curves are sufficient to verify the main numerical claims reported here. Exact end-to-end reproduction still requires reconstructing the cleaned Armenian corpus used during training.

VIII. LIMITATIONS

The results demonstrate a clear improvement in Armenian tokenization efficiency and a strong recovery of language-model likelihood after lightweight adaptation, but the empirical scope remains bounded. The strongest

supported claims are about fertility reduction, compression behavior, embedding-initialization effects, and held-out perplexity after LoRA recovery. The qualitative generation checks show that Armenian continuations remain weak or mixed: prompts are often followed by English, multilingual, or loosely related text. Consequently, the experiments should not be interpreted as establishing full downstream Armenian generation quality.

The evaluation also does not include a formal downstream Armenian task benchmark such as classification, named-entity recognition, question answering, or sequence labeling. This is a deliberate boundary of the present prototype: perplexity measures whether the adapted model assigns higher likelihood to Armenian text after tokenizer surgery, but it does not by itself measure task competence or instruction-following behavior. Downstream conclusions therefore remain limited until the same tokenizer and adapters are tested on labeled Armenian benchmarks.

Some reproduction steps depend on large corpus assets that were not bundled directly with the local submission materials. The completed experimental pipeline produced trained tokenizer files, adapter outputs, benchmark summaries, and qualitative evaluation artifacts, but the full cleaned training corpus `hy_clean.txt` was not included as a standalone file because of its large size. The corpus is described as a cleaned CC-100 Armenian dataset containing approximately 12.1 million lines, from which the training configuration sampled 500,000 filtered lines. Reproducing the full pipeline exactly would therefore require access to the same corpus or regeneration through the documented cleaning procedure.

Runtime comparisons should be treated as descriptive rather than methodological evidence. The heuristic-init run was trained on a Google Colab T4, the mean-init run on an H100 environment, and the final nearest-init run on an RTX PRO 6000 workstation-class GPU. These hardware differences confound wall-clock training time, so the meaningful comparisons are the tokenization metrics, pre-fine-tuning perplexity, post-fine-tuning perplexity, and training and evaluation loss trajectories rather than raw minutes.

IX. FUTURE WORK

A natural next step is longer and more controlled Armenian recovery training. Future runs should use a fixed compute environment, fixed train/evaluation splits, explicit corpus checksums, and matched optimization schedules across initialization strategies. Longer continued pretraining on the cleaned Armenian corpus, possibly with additional high-quality Armenian sources, would test whether the observed perplexity recovery continues to translate into more stable Armenian continuations.

The adapted models should also be evaluated on downstream Armenian tasks. Suitable benchmarks include topic or sentiment classification, named-entity recognition, morphological or part-of-speech tagging, extractive question

answering, and retrieval-augmented evaluation over Armenian passages. These tasks would clarify whether the improved tokenizer efficiency and lower perplexity produce measurable gains in practical NLP settings.

Generation quality requires a separate evaluation protocol. Future work should use curated Armenian prompts, automatic checks for Armenian-script retention and repetition, and human evaluation of fluency, factual coherence, and prompt adherence. Instruction tuning or supervised fine-tuning on Armenian question-answer and instruction-following data is likely necessary before the adapted model can be assessed as a usable Armenian assistant rather than a recovered causal language model.

Further ablations would strengthen the design conclusions. Vocabulary sizes beyond 8k, 16k, and 32k should be tested against context efficiency and model recovery cost; BPE and Unigram tokenizers should be compared under matched downstream training; and initialization variants should be evaluated with multiple seeds. Scaling the same procedure to larger or stronger base models would also show whether tokenizer surgery remains beneficial when the pretrained model has greater multilingual capacity.

The methodology can also be extended beyond Armenian. Georgian is a promising next target because it is likewise a low-resource language with its own script and rich morphological structure. A controlled Georgian extension would test whether tokenizer surgery and lightweight adaptation generalize across morphologically rich Caucasian languages, while also revealing which parts of the pipeline are language-specific and which are reusable.

Finally, reproducibility can be improved by packaging the corpus-preparation script, exact environment files, result-generation commands, model-card metadata, and benchmark summaries into a single release structure.

X. CONCLUSION

Tokenizer fragmentation is a serious bottleneck for Armenian NLP, and tokenizer surgery substantially reduces it in this setting. Standard English-centric tokenizers fragment Armenian severely: Qwen2.5-0.5B uses about 7.8 tokens per Armenian word, and GPT-2 uses more than 14. Custom Armenian SentencePiece tokenizers reduce fertility to 1.67–2.06 with zero UNK rate in the evaluation.

The grafting experiment shows that this tokenizer improvement can be attached to a small pretrained model by extending its vocabulary. Adding 30,766 Armenian-oriented tokens to Qwen reduced tokenization cost by 78.3% on the Goal 3 sample. Among initialization strategies, heuristic/FOCUS initialization was best before fine-tuning and remained best after LoRA recovery, reaching evaluation perplexity 8.33. These results support the central hypothesis that informed embedding initialization and lightweight adaptation can make tokenizer surgery practical for low-resource Armenian. The resulting model is best viewed as a successful research prototype for efficient Armenian tokenization and perplexity recovery,

with downstream generation and task performance left for controlled follow-up evaluation.

REFERENCES

- [1] K. Dobler and G. de Melo, “FOCUS: Effective embedding initialization for monolingual specialization of multilingual models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [2] T. Kudo and J. Richardson, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.
- [3] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [4] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, “mT5: A massively multilingual pre-trained text-to-text transformer,” *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 2021.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *International Conference on Learning Representations*, 2022.
- [6] Qwen Team, “Qwen2.5: A party of foundation models,” <https://qwenlm.github.io/>, 2025.
- [7] Armenian Tokenizer Surgery Project, “Goal 1: Tokenizer fertility analysis,” https://github.com/levongevorgian/Low-resource-Armenian-NLP/tree/main/goal_1_tokenizer_fertility, 2026, GitHub repository folder for Goal 1 experiments and reports.
- [8] —, “Goal 2: Custom armenian tokenizer training and evaluation,” https://github.com/levongevorgian/Low-resource-Armenian-NLP/tree/main/goal_2_tokenizer_surgery, 2026, GitHub repository folder for tokenizer training and evaluation.
- [9] —, “Goal 3: Armenian tokenizer grafting and embedding initialization,” https://github.com/levongevorgian/Low-resource-Armenian-NLP/tree/main/goal_3_lightweight_adaptation, 2026, GitHub repository folder for vocabulary grafting experiments.
- [10] —, “Goal 4: Lora recovery fine-tuning and evaluation,” https://github.com/levongevorgian/Low-resource-Armenian-NLP/tree/main/goal_4_evaluation, 2026, GitHub repository folder for LoRA recovery fine-tuning experiments.
- [11] —, “Goal 5: Final evaluation, generation diagnostics, and visualization,” https://github.com/levongevorgian/Low-resource-Armenian-NLP/tree/main/goal_5_generation_or_analysis, 2026, GitHub repository for consolidated metrics, generation tests, English preservation, and report figures.